

Instituto Tecnológico Superior del Oriente del Estado de Hidalgo

Métodos Numéricos

Docente: Dr. Efrén Rolando Romero León

Alumna: Naomi Anaís Sandoval Godinez

Evidencia: T6 Problemario: Solución Numérica de Ecuaciones Diferenciales

1. Método de Euler Básico

- **Descripción General:** Es el método más simple y el punto de partida para resolver Ecuaciones Diferenciales Ordinarias (EDO) de primer orden con un valor inicial. Utiliza la pendiente al inicio del intervalo para aproximar el siguiente punto de manera lineal.
- **Algoritmo:**
 1. Iniciar con los valores conocidos (x_0, y_0) .
 2. Calcular la pendiente de la curva en el punto actual usando la función dada $f(x, y)$.
 3. Multiplicar esta pendiente por el tamaño del paso h para hallar el incremento en y .
 4. Actualizar x sumándole h y repetir el proceso en un bucle.
- **Fórmula:**

$$y_{n+1} = y_n + h \cdot f(x_n, y_n)$$

CODIGO

```
public class MetodoEulerBasico {  
  
    // Definimos la función f(x, y) = dy/dx  
    public static double f(double x, double y) {  
        return x + y;  
    }  
  
    public static void main(String[] args) {  
        // Condiciones iniciales  
        double x0 = 0.0;  
        double y0 = 1.0;  
  
        double h = 0.1; // Tamaño del paso  
        double xObjetivo = 1.0; // Punto donde queremos aproximar y  
  
        double x = x0;  
        double y = y0;  
  
        System.out.println("--- METODO DE EULER ---");  
        System.out.printf("x = %.2f -> y = %.4f\n", x, y);  
  
        // Bucle para iterar desde x0 hasta xObjetivo  
        while (x < xObjetivo) {  
            // Fórmula de Euler: y_next = y + h * f(x, y)  
            y = y + h * f(x, y);  
            x = x + h; // Avanzamos el paso  
  
            System.out.printf("x = %.2f -> y = %.4f\n", x, y);  
        }  
  
        System.out.println("-----");  
        System.out.printf("Resultado aproximado: y(%.1f) = %.4f\n", xObjetivo, y);  
    }  
}
```

COMPILACIÓN

```
--- METODO DE EULER ---
```

```
x = 0.00 -> y = 1.0000
```

```
x = 0.10 -> y = 1.1000
```

```
x = 0.20 -> y = 1.2200
```

```
x = 0.30 -> y = 1.3620
```

```
x = 0.40 -> y = 1.5282
```

```
x = 0.50 -> y = 1.7210
```

```
x = 0.60 -> y = 1.9431
```

```
x = 0.70 -> y = 2.1974
```

```
x = 0.80 -> y = 2.4872
```

```
x = 0.90 -> y = 2.8159
```

```
x = 1.00 -> y = 3.1875
```

```
x = 1.10 -> y = 3.6062|
```

```
-----
```

```
Resultado aproximado: y(1.0) = 3.6062
```

```
=== Code Execution Successful ===
```

2. Método de Adams-Bashforth (2 Pasos)

- **Descripción General:** Es un método **multipaso explícito**. A diferencia de Euler, que solo mira el punto actual, este método utiliza la información de la pendiente de los dos puntos anteriores (x_n y x_{n-1}) para trazar una aproximación mucho más precisa.
- **Algoritmo:**
 - 1 Al requerir dos puntos iniciales, el paso 0 se toma de la condición inicial y el paso 1 se calcula con un método de un solo paso (como Euler simple).
 - 2 Una vez que se tienen ambos puntos, se evalúa la función f en ambos extremos históricos.
 - 3 Se aplica la combinación ponderada de ambas pendientes para proyectar el siguiente valor y_{n+1} .
- **Fórmula:**

$$y_{n+1} = y_n + \frac{h}{2} \cdot [3f(x_n, y_n) - f(x_{n-1}, y_{n-1})]$$

CODIGO

```
AdamsBashforth2Pasos.java
public static double f(double x, double y) {
    return x + (2 * y);
}

public static void main(String[] args) {
    double h = 0.1;
    int numPasos = 5;

    // Arreglos para almacenar los puntos x e y
    double[] x = new double[numPasos + 1];
    double[] y = new double[numPasos + 1];

    // Paso 0: Condición inicial
    x[0] = 0.0;
    y[0] = 1.0;

    // Paso 1: Usamos Euler simple para obtener el segundo punto necesario
    x[1] = x[0] + h;
    y[1] = y[0] + h * f(x[0], y[0]);

    System.out.println("--- MÉTODO DE ADAMS-BASHFORTH (2 PASOS) ---");
    System.out.printf("Paso 0: x = %.1f -> y = %.5f (Inicial)%n", x[0], y[0]);
    System.out.printf("Paso 1: x = %.1f -> y = %.5f (Por Euler)%n", x[1], y[1]);

    // Pasos del 2 en adelante usando Adams-Bashforth de 2 pasos
    for (int n = 1; n < numPasos; n++) {
        x[n+1] = x[n] + h;

        // Fórmula:  $y_{n+1} = y_n + (h/2) * [3*f(x_n, y_n) - f(x_{n-1}, y_{n-1})]$ 
        y[n+1] = y[n] + (h / 2.0) * (3 * f(x[n], y[n]) - f(x[n-1], y[n-1]));

        System.out.printf("Paso %d: x = %.1f -> y = %.5f (Por Adams-Bashforth)%n", n+1,
            x[n+1], y[n+1]);
    }
}
```

COMPILACIÓN

```
^ --- M?TODO DE ADAMS-BASHFORTH (2 PASOS) ---  
Paso 0: x = 0.0 -> y = 1.00000 (Inicial)  
Paso 1: x = 0.1 -> y = 1.20000 (Por Euler)  
Paso 2: x = 0.2 -> y = 1.47500 (Por Adams-Bashforth)  
Paso 3: x = 0.3 -> y = 1.82250 (Por Adams-Bashforth)  
Paso 4: x = 0.4 -> y = 2.25675 (Por Adams-Bashforth)  
Paso 5: x = 0.5 -> y = 2.79653 (Por Adams-Bashforth)  
  
=== Code Execution Successful ===
```

3. Ley de Enfriamiento de Newton (Aplicación de Euler)

- **Descripción General:** No es un método numérico nuevo, sino la **aplicación práctica** del Método de Euler para modelar un fenómeno físico: cómo cambia la temperatura de un objeto (café) en un ambiente constante. La tasa de pérdida de calor es proporcional a la diferencia de temperatura entre el objeto y el ambiente.
- **Algoritmo:**
 - 1 Definir la temperatura ambiente (T_m), la constante de enfriamiento (k) y la temperatura inicial del objeto (T_0).
 - 2 En cada intervalo de tiempo (h), calcular el cambio térmico instantáneo ($dT = -k \cdot (T - T_m)$).
 - 3 Actualizar la temperatura sumando el cambio multiplicado por el paso del tiempo: $T = T + h \cdot dT$.
- **Fórmula:**

$$\frac{dT}{dt} = -k(T - T_{ambiente}) \quad \rightarrow \quad T_{n+1} = T_n + h \cdot [-k(T_n - T_{ambiente})]$$

CODIGO

```
1 public class EnfriamientoNewton {
2
3     public static void main(String[] args) {
4         // Datos del problema de aplicación
5         double tempAmbiente = 20.0; // Tm (Habitación climatizada a 20°C)
5         double k = 0.05;           // Constante de pérdida de calor del material por minuto
7
8         double t = 0.0;           // Tiempo inicial (minutos)
9         double T = 80.0;          // Temperatura inicial del café en °C
10        double h = 2.0;           // Monitorear cada 2 minutos
11        double tiempoTotal = 20.0; // Simular por 20 minutos
12
13        System.out.println("--- SIMULACIÓN DE INGENIERÍA: ENFRIAMIENTO DE CAFÉ ---");
14        System.out.println("-----");
15        System.out.printf("%-15s | %-20s\n", "Tiempo (min)", "Temperatura (°C)");
16        System.out.println("-----");
17
18        while (t <= tiempoTotal) {
19            System.out.printf("%-15.1f | %-20.2f\n", t, T);
20
21            // Cálculo del cambio térmico instantáneo dT/dt
22            double dT = -k * (T - tempAmbiente);
23
24            // Avance al paso siguiente empleando el método de Euler
25            T = T + h * dT;
26            t = t + h;
27        }
28        System.out.println("-----");
29    }
30 }
```

COMPILACIÓN

```
--- SIMULACIÓN DE INGENIERÍA: ENFRIAMIENTO DE CAFÉ ---
```

```
-----  
Tiempo (min) | Temperatura (°C)  
-----
```

0.0	80.00
2.0	74.00
4.0	68.60
6.0	63.74
8.0	59.37
10.0	55.43
12.0	51.89
14.0	48.70
16.0	45.83
18.0	43.25
20.0	40.92

```
-----  
  
=== Code Execution Successful ===
```

4. Método de Heun (Euler Mejorado)

- **Descripción General:** Es un método de tipo **Predictor-Corrector** de un solo paso. Resuelve el problema de Euler (que solo usa la pendiente inicial) calculando una pendiente promedio entre el inicio y una estimación del final del intervalo.
- **Algoritmo:**
 - 1 **Predicción:** Se calcula un valor tentativo de y para el siguiente punto ($\hat{y}_{n+1}$) usando la fórmula de Euler estándar.
 - 2 **Corrección:** Se calcula la pendiente en el punto actual y la pendiente en el punto aproximado del futuro. Se saca el promedio de ambas pendientes.
 - 3 Se usa este promedio para calcular el valor definitivo de y_{n+1} .
- **Fórmula:**

$$\text{Predicor: } \hat{y}_{n+1} = y_n + h \cdot f(x_n, y_n)$$

$$\text{Corrector: } y_{n+1} = y_n + \frac{h}{2} \cdot [f(x_n, y_n) + f(x_{n+1}, \hat{y}_{n+1})]$$

CODIGO

```
1 public class MetodoHeun {
2
3     public static double f(double x, double y) {
4         return (x * x) - y;
5     }
6
7     public static void main(String[] args) {
8         double x = 0.0;
9         double y = 1.0;
10        double h = 0.1;
11        double xObjetivo = 0.5;
12
13        System.out.println("--- METODO DE HEUN (EULER MEJORADO) ---");
14        System.out.printf("x = %.1f -> y = %.5f\n", x, y);
15
16        while (x < xObjetivo - 0.0001) {
17            // 1. Predictor (Euler estándar)
18            double yPredictor = y + h * f(x, y);
19
20            // 2. Corrector (Promedio de pendientes)
21            double pendienteInicial = f(x, y);
22            double pendienteSiguiente = f(x + h, yPredictor);
23
24            y = y + (h / 2.0) * (pendienteInicial + pendienteSiguiente);
25            x = x + h;
26
27            System.out.printf("x = %.1f -> y = %.5f\n", x, y);
28        }
29    }
```

COMPILACIÓN

```
--- METODO DE HEUN (EULER MEJORADO) ---
```

```
x = 0.0 -> y = 1.00000
```

```
x = 0.1 -> y = 0.90550
```

```
x = 0.2 -> y = 0.82193
```

```
x = 0.3 -> y = 0.75014
```

```
x = 0.4 -> y = 0.69093
```

```
x = 0.5 -> y = 0.64499
```

```
=== Code Execution Successful ===
```

5. Simulación de Péndulo (Sistema de EDO de 2do Orden)

- **Descripción General:** Este programa modela un sistema físico complejo transformando una ecuación diferencial de **segundo orden** (aceleración angular del péndulo) en un sistema de dos ecuaciones acopladas de **primer orden** (una para la velocidad y otra para la posición).
- **Algoritmo:**
 - 1 Definir las variables de estado simultáneas: el ángulo (θ) y la velocidad angular (ω).
 - 2 Calcular las derivadas simultáneamente: la derivada del ángulo es la velocidad, y la derivada de la velocidad viene dada por la gravedad y la longitud ($-\frac{g}{L} \sin(\theta)$).
 - 3 Actualizar ambas variables al mismo tiempo usando el método de Euler antes de pasar al siguiente paso de tiempo.
- **Fórmula:**

$$\frac{d\theta}{dt} = \omega, \quad \frac{d\omega}{dt} = -\frac{g}{L} \sin(\theta)$$

$$\theta_{n+1} = \theta_n + h \cdot \omega_n, \quad \omega_{n+1} = \omega_n + h \cdot \left(-\frac{g}{L} \sin(\theta_n)\right)$$

CODIGO

```
PendoloSistema.java    Share  Run
```

```
1 public class PendoloSistema {
2
3     public static void main(String[] args) {
4         double t = 0.0;
5         double h = 0.05; // Paso de tiempo en segundos
6         double tMax = 1.0;
7
8         // Parámetros físicos
9         double g = 9.81; // Gravedad
10        double L = 1.0; // Longitud de la cuerda (1 metro)
11
12        // Condiciones iniciales
13        double theta = 0.5; // Ángulo inicial en radianes (~28 grados)
14        double omega = 0.0; // Velocidad angular inicial (se suelta desde el reposo)
15
16        System.out.println("--- SIMULACION DE PENDULO (SISTEMA DE EDO DE 2DO ORDEN) ---");
17        System.out.printf("Tiempo: %.2fs -> Ángulo: %.4f rad | Vel. Angular: %.4f rad/s\n",
18            , theta, omega);
19
20        while (t < tMax - 0.001) {
21            // Calcular derivadas simultáneas
22            double dTheta = omega;
23            double dOmega = -(g / L) * Math.sin(theta);
24
25            // Actualización simultánea por el método de Euler
26            theta = theta + h * dTheta;
27            omega = omega + h * dOmega;
28            t = t + h;
29
30            System.out.printf("Tiempo: %.2fs -> Ángulo: %.4f rad | Vel. Angular: %.4f rad
31                /s\n", t, theta, omega);
32        }
33    }
34 }
```

COMPILACIÓN

```
--- SIMULACION DE PENDULO (SISTEMA DE EDO DE 2DO ORDEN) ---  
Tiempo: 0.00s -> Angulo: 0.5000 rad | Vel. Angular: 0.0000 rad/s  
Tiempo: 0.05s -> Angulo: 0.5000 rad | Vel. Angular: -0.2352 rad/s  
Tiempo: 0.10s -> Angulo: 0.4882 rad | Vel. Angular: -0.4703 rad/s  
Tiempo: 0.15s -> Angulo: 0.4647 rad | Vel. Angular: -0.7004 rad/s  
Tiempo: 0.20s -> Angulo: 0.4297 rad | Vel. Angular: -0.9202 rad/s  
Tiempo: 0.25s -> Angulo: 0.3837 rad | Vel. Angular: -1.1246 rad/s  
Tiempo: 0.30s -> Angulo: 0.3275 rad | Vel. Angular: -1.3082 rad/s  
Tiempo: 0.35s -> Angulo: 0.2621 rad | Vel. Angular: -1.4660 rad/s  
Tiempo: 0.40s -> Angulo: 0.1888 rad | Vel. Angular: -1.5930 rad/s  
Tiempo: 0.45s -> Angulo: 0.1091 rad | Vel. Angular: -1.6851 rad/s  
Tiempo: 0.50s -> Angulo: 0.0249 rad | Vel. Angular: -1.7385 rad/s  
Tiempo: 0.55s -> Angulo: -0.0621 rad | Vel. Angular: -1.7507 rad/s  
Tiempo: 0.60s -> Angulo: -0.1496 rad | Vel. Angular: -1.7202 rad/s  
Tiempo: 0.65s -> Angulo: -0.2356 rad | Vel. Angular: -1.6471 rad/s  
Tiempo: 0.70s -> Angulo: -0.3180 rad | Vel. Angular: -1.5326 rad/s  
Tiempo: 0.75s -> Angulo: -0.3946 rad | Vel. Angular: -1.3793 rad/s  
Tiempo: 0.80s -> Angulo: -0.4636 rad | Vel. Angular: -1.1907 rad/s  
Tiempo: 0.85s -> Angulo: -0.5231 rad | Vel. Angular: -0.9714 rad/s  
Tiempo: 0.90s -> Angulo: -0.5717 rad | Vel. Angular: -0.7263 rad/s  
Tiempo: 0.95s -> Angulo: -0.6080 rad | Vel. Angular: -0.4610 rad/s  
Tiempo: 1.00s -> Angulo: -0.6310 rad | Vel. Angular: -0.1808 rad/s  
  
=== Code Execution Successful ===
```


6. Método Predictor-Corrector (Adams-Bashforth-Moulton)

- **Descripción General:** Combina la naturaleza multipaso explícita (Adams-Bashforth) con una corrección implícita (Adams-Moulton). Esto permite obtener una estabilidad matemática y precisión drásticamente superiores al trabajar con varios puntos previos.
- **Algoritmo:**
 - 1 Se inicializan los primeros puntos usando un método de un paso (como Runge-Kutta 2).
 - 2 **Predictor:** Se calcula un valor estimado del futuro ($\hat{y}_{n+1}$) usando la fórmula de Adams-Bashforth de 2 pasos.
 - 3 **Corrector:** Se evalúa la función en este nuevo punto hipotético y se refina el resultado usando la fórmula implícita de Adams-Moulton.
- **Fórmula:**

$$\text{Predicor (AB2): } \hat{y}_{n+1} = y_n + \frac{h}{2}[3f(x_n, y_n) - f(x_{n-1}, y_{n-1})]$$

$$\text{Corrector (AM2): } y_{n+1} = y_n + \frac{h}{2}[f(x_{n+1}, \hat{y}_{n+1}) + f(x_n, y_n)]$$

CODIGO

```
predictorCorrector.java    Share  Run
```

```
3 ~ public static double f(double x, double y) {
4     return y - (x * x) + 1;
5 }
6
7 ~ public static void main(String[] args) {
8     double h = 0.1;
9     int pasos = 4;
10    double[] x = new double[pasos + 1];
11    double[] y = new double[pasos + 1];
12
13    // Paso 0: Iniciales
14    x[0] = 0.0; y[0] = 0.5;
15
16    // Paso 1: Arrancamos con Runge-Kutta 2 (Heun) para inicializar el historial
17    x[1] = x[0] + h;
18    double k1 = f(x[0], y[0]);
19    double k2 = f(x[1], y[0] + h*k1);
20    y[1] = y[0] + (h/2.0) * (k1 + k2);
21
22    System.out.println("--- METODO PREDICTOR-CORRECTOR ---");
23    System.out.printf("Paso 0: x = %.1f -> y = %.5f%n", x[0], y[0]);
24    System.out.printf("Paso 1: x = %.1f -> y = %.5f (Por RK2)%n", x[1], y[1]);
25
26    // Ciclo Predictor-Corrector para los siguientes pasos
27 ~ for (int n = 1; n < pasos; n++) {
28     x[n+1] = x[n] + h;
29
30     // 1. PREDICTOR: Adams-Bashforth 2 pasos
31     double yPredicho = y[n] + (h / 2.0) * (3 * f(x[n], y[n]) - f(x[n-1], y[n-1]));
32
33     // 2. CORRECTOR: Adams-Moulton 2 pasos (usa el valor predicho en x_{n+1})
34     y[n+1] = y[n] + (h / 2.0) * (f(x[n+1], yPredicho) + f(x[n], y[n]));
35
36     System.out.printf("Paso %d: x = %.1f -> y = %.5f%n", n+1, x[n+1], y[n+1]);
37 }
38 }
```

COMPILACIÓN

```
--- METODO PREDICTOR-CORRECTOR ---
```

```
Paso 0: x = 0.0 -> y = 0.50000
```

```
Paso 1: x = 0.1 -> y = 0.65700 (Por RK2)
```

```
Paso 2: x = 0.2 -> y = 0.82880
```

```
Paso 3: x = 0.3 -> y = 1.01448
```

```
Paso 4: x = 0.4 -> y = 1.21339
```

```
=== Code Execution Successful ===
```

7. Método de Runge-Kutta de 4º Orden (RK4)

- **Descripción General:** Es el estándar de oro en los métodos numéricos de un solo paso debido a su **extrema precisión**. Logra la exactitud de una serie de Taylor de cuarto orden calculando cuatro pendientes distintas (k_1, k_2, k_3, k_4) en diferentes partes del intervalo y promediándolas de forma ponderada.
- **Algoritmo:**
 - 1 Calcular k_1 : la pendiente al inicio del intervalo.
 - 2 Calcular k_2 : la pendiente en el punto medio del intervalo usando k_1 .
 - 3 Calcular k_3 : otra pendiente en el punto medio usando k_2 .
 - 4 Calcular k_4 : la pendiente al final del intervalo usando k_3 .
 - 5 Sumar a y_n un promedio ponderado donde los puntos medios (k_2, k_3) tienen el doble de peso.
- **Fórmula:**

$$k_1 = f(x_n, y_n), \quad k_2 = f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right)$$

$$k_3 = f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}k_2\right), \quad k_4 = f(x_n + h, y_n + h \cdot k_3)$$

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

CODIGO

```
RungeKutta4.java    Share  Run
```

```
1 public class RungeKutta4 {
2
3     // Función f(x, y) = dy/dx
4     public static double f(double x, double y) {
5         return x * Math.sqrt(y);
6     }
7
8     public static void main(String[] args) {
9         double x = 1.0; // Condición inicial x0
10        double y = 1.0; // Condición inicial y0
11        double h = 0.1; // Tamaño de paso
12        double xObjetivo = 1.5;
13
14        System.out.println("--- METODO DE RUNGE-KUTTA DE 4º ORDEN (RK4) ---");
15        System.out.printf("x = %.1f -> y = %.6f\n", x, y);
16
17        while (x < xObjetivo - 0.0001) {
18            // Calcular las 4 pendientes intermedias
19            double k1 = f(x, y);
20            double k2 = f(x + h/2.0, y + (h*k1)/2.0);
21            double k3 = f(x + h/2.0, y + (h*k2)/2.0);
22            double k4 = f(x + h, y + h*k3);
23
24            // Actualizar y usando el promedio ponderado
25            y = y + (h / 6.0) * (k1 + 2*k2 + 2*k3 + k4);
26            x = x + h;
27
28            System.out.printf("x = %.1f -> y = %.6f\n", x, y);
29        }
30    }
31 }
```

COMPILACIÓN

```
--- METODO DE RUNGE-KUTTA DE 4º ORDEN (RK4) ---
```

```
x = 1.0 -> y = 1.000000
```

```
x = 1.1 -> y = 1.107756
```

```
x = 1.2 -> y = 1.232100
```

```
x = 1.3 -> y = 1.374756
```

```
x = 1.4 -> y = 1.537600
```

```
x = 1.5 -> y = 1.722656
```

```
=== Code Execution Successful ===
```

8. Simulación Sistema de EDOs (Modelo Depredador-Presa)

- **Descripción General:** Aplica el método numérico para resolver un sistema interactivo de EDOs no lineales conocido como las ecuaciones de **Lotka-Volterra**. Modela la evolución de dos poblaciones independientes (conejos/presas y zorros/depredadores) que dependen mutuamente entre sí.
- **Algoritmo:**
 - 1 Establecer las poblaciones iniciales de presas (x) y depredadores (y).
 - 2 Calcular la tasa de cambio de las presas (dX) considerando su tasa de nacimiento menos la tasa de encuentros destructivos con depredadores.
 - 3 Calcular la tasa de cambio de los depredadores (dY) considerando su tasa de mortalidad natural contrarrestada por el alimento que consiguen de las presas.
 - 4 **Crucial:** Actualizar ambas poblaciones de manera estrictamente **simultánea** usando los valores del paso actual antes de avanzar el tiempo.
- **Fórmula:**

$$\frac{dx}{dt} = x \cdot (\alpha - \beta y), \quad \frac{dy}{dt} = y \cdot (-\gamma + \delta x)$$

$$x_{n+1} = x_n + h \cdot dX(x_n, y_n)$$

CODIGO

```
SystemEDOS.java

4- public static double dX_dt(double x, double y) {
5-     return x - (0.1 * x * y);
6- }
7
8- // Cambios en la población de Depredadores (y)
9- public static double dY_dt(double x, double y) {
10-     return -0.5 * y + (0.02 * x * y);
11- }
12
13- public static void main(String[] args) {
14-     double t = 0.0;
15-     double h = 0.2; // Paso de tiempo
16-     double tMax = 2.0;
17
18-     // Poblaciones iniciales
19-     double x = 40.0; // Ejemplo: 40 conejos
20-     double y = 9.0; // Ejemplo: 9 zorros
21
22-     System.out.println("--- SIMULACION SISTEMA DE EDOS (Depredador-Presa) ---");
23-     System.out.printf("Tiempo: %.1f -> Presas: %.2f | Depredadores: %.2f\n", t, x, y);
24
25-     while (t < tMax - 0.001) {
26-         // CRUCIAL: Calcular los cambios usando los valores del paso actual
27-         double cambioX = dX_dt(x, y);
28-         double cambioY = dY_dt(x, y);
29
30-         // Actualizar simultáneamente
31-         x = x + h * cambioX;
32-         y = y + h * cambioY;
33-         t = t + h;
34
35-         System.out.printf("Tiempo: %.1f -> Presas: %.2f | Depredadores: %.2f\n", t, x,
36-             y);
37-     }
38- }
```


COMPILACIÓN

```
--- SIMULACION SISTEMA DE EDOS (Depredador-Presa) ---
```

```
Tiempo: 0.0 -> Presas: 40.00 | Depredadores: 9.00
```

```
Tiempo: 0.2 -> Presas: 40.80 | Depredadores: 9.54
```

```
Tiempo: 0.4 -> Presas: 41.18 | Depredadores: 10.14
```

```
Tiempo: 0.6 -> Presas: 41.06 | Depredadores: 10.80
```

```
Tiempo: 0.8 -> Presas: 40.40 | Depredadores: 11.49
```

```
Tiempo: 1.0 -> Presas: 39.20 | Depredadores: 12.20
```

```
Tiempo: 1.2 -> Presas: 37.47 | Depredadores: 12.89
```

```
Tiempo: 1.4 -> Presas: 35.30 | Depredadores: 13.54
```

```
Tiempo: 1.6 -> Presas: 32.80 | Depredadores: 14.09
```

```
Tiempo: 1.8 -> Presas: 30.12 | Depredadores: 14.53
```

```
Tiempo: 2.0 -> Presas: 27.39 | Depredadores: 14.83
```

```
=== Code Execution Successful ===
```

En conclusión, la progresión de estos métodos demuestra cómo el análisis numérico evoluciona desde la aproximación lineal simple de Euler hacia algoritmos de alta precisión como Runge-Kutta (RK4) y sistemas eficientes de predicción y corrección (Adams-Bashforth-Moulton). Su verdadera importancia radica en la capacidad de trasladar las matemáticas abstractas a la modelación del mundo real, permitiendo simular con rigor científico el enfriamiento térmico, la mecánica de un péndulo o el equilibrio ecológico entre especies.